Object Oriented Programming (Lab)

BSIT - Fall 2024

Dr. Muhammad Farooq

The objective of this lab is to:
To understand and utilize the concept of arrays (1D and 2D) along with the previous concepts.

ALERT!

1. This is an individual lab. You are **strictly NOT allowed to collaborate** with others, share screens, or communicate answers in any form.
2. Use of **AI tools (e.g., ChatGPT, Copilot, etc.) is strictly prohibited**. Any AI-generated content will be treated as academic dishonesty.
3. Anyone caught in the act of **cheating would be awarded a 0** in this Lab.
4. MAKE SURE TO **VALIDATE WHERE EVER YOU TAKE INPUT** FROM THE USER< OTHERWISE MARKS SHALL BE DEDUCTED.

Lab - 6

Task 01 (10 marks)

Implement a class with following definition:

```
class Book
{
   string title;
   string author;
   int publicationYear;
   double price;
public:
   // TODO: Implement getter and setter methods for each data member
   // TODO: Implement a parameterized constructor to initialize all data members
   // TODO: Implement this function to display book details
   void displayBookInfo();
   // TODO: Implement this function to check if the book is antique (>= 50 years old)
   bool isAntique(int currentYear);
};
```

Your task is to:

1. Write getter and setter methods for all private data members.
2. Implement a parameterized constructor that initializes title, author, publicationYear, and price.
3. Implement:
    - displayBookInfo() → prints all book details.
    - isAntique(int currentYear) → returns true if the book is at least 50 years old (based on currentYear), otherwise false.

Input:
Enter Book Title: The Alchemist
Enter Author: Paulo Coelho
Enter Publication Year: 1988
Enter Price: 15.99
Enter Current Year: 2023

Output:
Title: The Alchemist
Author: Paulo Coelho
Year: 1988
Price: $15.99
The book is not antique.

Task 02 (10 marks)

You are required to write a C++ program that dynamically allocates memory for a 2D array of integers and performs matrix transposition without using any built-in functions or additional 2D arrays.

Your program should:
1. Ask the user for the number of rows and columns.
2. Dynamically allocate memory for the 2D array using pointer-to-pointer.
3. Input the matrix elements from the user.
4. Display the original matrix.
5. Transpose the matrix **in-place** (without creating another matrix).
6. Display the transposed matrix.

Use the following function to perform in-place transposition (for square matrices):

void transposeMatrix(int matrix, int size);**

For non-square matrices, display: **"In-place transposition not possible for non-square matrix."**

Input:
Enter rows and columns: 3 3
Enter matrix elements:
1 2 3
4 5 6
7 8 9

Output:
Original Matrix:
1 2 3
4 5 6
7 8 9

Transposed Matrix:
1 4 7
2 5 8
3 6 9

Task 03 (10 marks)

Write a C++ program to manage student exam records using file handling.

The program should:
1. **Input Phase**
   - Ask the user for the number of students **N**.
   - Dynamically allocate arrays to store:
     - Student Names
     - Roll Numbers
     - Marks in 3 subjects (Math, Science, English)

2. **File Storage Phase**
   Write all data into a file named `**marks.txt**` in the following format:

Name RollNo Math Science English
Ali 101 85 90 78
Sara 102 92 88 95

3. **Processing Phase**
- Read data from `marks.txt`.
- Calculate for each student:
    - Total marks
    - Average marks
    - Grade (A: >=80, B: >=70, C: >=60, F: <60)
- Write the results into a new file `results.txt` with the format:

Name RollNo Total Average Grade
Ali 101 253 84.33 A

4. **Display Phase**
Read and display the contents of **`results.txt`** on the console.
Also display the student with the highest total marks.

Example Execution:

Enter number of students: 2
Enter details for student 1:
Name: Ali
Roll Number: 101
Marks (Math Science English): 85 90 78

---- Results ----
Name: Ali | Roll No: 101 | Total: 253 | Average: 84.33 | Grade: A
Name: Sara | Roll No: 102 | Total: 275 | Average: 91.67 | Grade: A
Highest Scorer: Sara (275)

Task 04 (10 marks)

Implement a class **`Student`** and create a dynamic array of `Student` objects to manage student records.

```
class Student
{
    string name;
    int rollNumber;
    double marks[3]; // marks in 3 subjects
public:
    // TODO: Implement getter and setter methods
    // TODO: Implement this function to input student details
    void inputStudentData();
    // TODO: Implement this function to calculate and return total marks
    double calculateTotal();
    // TODO: Implement this function to display student details and total marks
    void displayStudentData();
};
```

Your program should:
1. Ask the user for the number of students `N`.
2. Dynamically create an array of `N` `Student` objects.
3. For each student, call `inputStudentData()` to read details from the user.
4. Display all student details along with their total marks.
5. Find and display the student with the highest total marks.

Input:

```
Enter number of students: 2
Enter details for student 1:
Name: Ali
Roll Number: 101
Marks (3 subjects): 85 90 78
Enter details for student 2:
Name: Sara
Roll Number: 102
Marks (3 subjects): 92 88 95
```

Output:

---- Student Records ----
Name: Ali | Roll No: 101 | Marks: 85, 90, 78 | Total: 253
Name: Sara | Roll No: 102 | Marks: 92, 88, 95 | Total: 275
Highest Scorer: Sara (Total: 275)